

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

C02: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Debugging or Optimization task

Code of Conduct:

I ANISH K/192521288 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.

You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB

- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Q6. Semaphore Synchronization Debugging

A producer-consumer application occasionally hangs during execution. The semaphore values observed are:

mutex = 1

empty = 0

full = 10

Buffer size = 10

Producer Code:

wait(mutex)

wait(empty)

produce item

signal(full)

signal(mutex)

Consumer Code:

wait(mutex)

wait(full)

consume item

signal(empty)

signal(mutex)

Identify the synchronization bug causing the system to hang.

Explain which deadlock condition is being violated.

Suggest an optimization to ensure correct synchronization without changing the producer-consumer functionality. (10 Marks)

Q7. Thrashing Detection and Optimization

A virtual memory system reports the following statistics:

Number of processes: 50

Average page fault rate: 250 faults/sec

CPU Utilization: 18%

Disk Utilization: 95%

Free Frames Available: 2%

The operating system suspects thrashing.

Debug the root cause of the performance degradation.

Explain the relationship between page faults and CPU utilization.

Propose two operating system optimizations to reduce thrashing. (10 Marks)

Q8. File System Performance Debugging

A database server experiences slow file access times. System logs indicate:

Average file size: 500 MB

Disk seeks per request: 120

Disk utilization: 92%

Cache hit ratio: 15%

Identify the file system bottleneck affecting performance.

Explain why disk access latency is increasing.

Suggest two optimizations to improve file access efficiency. (10 Marks)

Q9. Thread Scheduling Debugging

A web server creates one thread per client request. During peak hours:

Active Threads: 5,000

CPU Utilization: 60%

Memory Usage: 92%

Average Response Time: 12 seconds

Scheduler logs show frequent thread creation and termination.

Identify the primary issue affecting server performance.

Explain how thread management contributes to the delay.

Recommend two optimizations to improve scalability and response time. (10 Marks)

Q10. Disk Scheduling Optimization Problem

A disk scheduling log shows the following request queue:

98, 183, 37, 122, 14, 124, 65, 67

Current Head Position: 53

The system uses FCFS scheduling and reports:

Average seek time: Very High

Throughput: Low

User complaints regarding slow responseDebug why FCFS is causing poor performance in this scenario.

Compare FCFS with one suitable disk scheduling algorithm for this workload.

Propose an optimization to improve seek time and overall throughput. (10 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Q1. Deadlock Debugging (10 Marks)

Problem Identification

A deadlock occurs because:

P1 locks R1 and waits for R2.

P2 locks R2 and waits for R1.

Each process is waiting for the resource held by the other.

This creates a Circular Wait, which is one of the four necessary deadlock conditions.

Root Cause

The processes acquire resources in different orders.

$P1 \rightarrow R1 \rightarrow R2$

$P2 \rightarrow R2 \rightarrow R1$

Since neither process releases its first resource, both remain blocked forever.

Optimization

Use Resource Ordering.

Both processes should acquire resources in the same order.

Example:

Process P1:

lock(R1)

lock(R2)

Critical Section

unlock(R2)

unlock(R1)

Process P2:

lock(R1)

lock(R2)

Critical Section

unlock(R2)

unlock(R1)

Result

Circular wait is eliminated.

Deadlock is prevented.

Process logic remains unchanged.

Q2. CPU Utilisation Problem (10 Marks)

Problem Identification

CPU utilization is only 30%, although processes are ready.

The scheduler log repeatedly shows:

Ready → Waiting → Ready → Waiting

Root Cause

Processes are I/O bound.

They frequently wait for I/O operations instead of using the CPU.

As a result:

CPU stays idle.

Many context switches occur.

Overall utilization decreases.

Optimization

1. Improve I/O Performance

Use buffering or caching.

Reduce waiting time.

2. Better CPU Scheduling

Use Multilevel Feedback Queue (MLFQ) or Round Robin.

Allow CPU-bound processes to execute while I/O-bound processes wait.

Result

CPU utilization increases.

Waiting time decreases.

System throughput improves.

Q3. Memory Bottleneck Debugging (10 Marks)

Problem Identification

Memory usage:

Code = 5 MB

Heap = 150 MB

Stack = 25 MB (1 MB $\times$ 25 threads)

The application experiences frequent page faults.

Root Cause

Large heap allocation and many threads increase memory consumption.

Frequently accessed pages are swapped between RAM and disk, causing excessive page faults.

Optimizations

1. Reduce Number of Threads

Use a thread pool instead of creating many threads.

2. Optimize Heap Usage

Free unused memory.

Allocate memory efficiently.

Result

Fewer page faults.

Lower memory usage.

Faster execution.

Q4. Starvation Debugging (10 Marks)

Problem Identification

Process P5 remains in Q3 (Batch Queue) and never gets CPU time.

Root Cause

Higher priority queues (Q1 and Q2) continuously receive processes.

Therefore, Q3 is never scheduled.

This leads to Starvation.

Optimization

Use Aging.

Increase the priority of processes waiting for a long time.

Example:

After waiting for a specific period,

$Q3 \rightarrow Q2 \rightarrow Q1$

Result

Every process eventually receives CPU time.

Fair scheduling is achieved.

Starvation is eliminated.

Q5. Context Switch Overhead Debugging (10 Marks)

Problem Identification

Context switches = 20,000/sec

Time quantum = 1 ms

Burst time = 2 ms

Root Cause

The quantum is too small.

Each thread is interrupted before finishing execution.

High I/O activity also causes additional context switches.

Optimizations

1. Increase Time Quantum

Example:

1 ms $\rightarrow$ 4 ms

This reduces unnecessary switching.

2. Use Thread Pooling

Reuse existing threads instead of creating new ones.

Result

Fewer context switches.

Lower CPU overhead.

Better throughput.

Q6. Semaphore Synchronization Debugging (10 Marks)

Problem Identification

The producer and consumer execute:

wait(mutex)

wait(empty/full)

Root Cause

The mutex is locked before checking resource availability.

If the buffer is full or empty:

Process blocks.

Mutex remains locked.

Other process cannot enter.

This causes deadlock.

Deadlock Condition

Hold and Wait

A process holds the mutex while waiting for another resource.

Optimization

Correct order:

Producer

wait(empty)

wait(mutex)

Produce

signal(mutex)

signal(full)

Consumer

wait(full)

wait(mutex)

Consume

signal(mutex)

signal(empty)

Result

No deadlock.

Proper synchronization.

Producer-consumer functionality remains unchanged.

Q7. Thrashing Detection and Optimization (10 Marks)

Problem Identification

Given:

Page Fault Rate = 250 faults/sec

CPU Utilization = 18%

Disk Utilization = 95%

Free Frames = 2%

Root Cause

The system is experiencing Thrashing.

Too many page faults force continuous swapping between RAM and disk.

Relationship

More page faults

↓

More disk access

↓

CPU waits longer

↓

CPU utilization decreases.

Optimizations

1. Reduce Degree of Multiprogramming

Run fewer processes simultaneously.

2. Allocate More Memory

Increase available frames or RAM.

Result

Page faults decrease.

CPU utilization increases.

Overall performance improves.

Q8. File System Performance Debugging (10 Marks)

Problem Identification

File size = 500 MB

Disk seeks = 120/request

Cache hit ratio = 15%

Root Cause

Frequent disk seeks and poor caching increase file access time.

Why Latency Increases

The disk head moves repeatedly to different locations.

More seeks mean longer access time.

Low cache hits force repeated disk reads.

Optimizations

1. Improve Disk Cache

Increase cache size to improve hit ratio.

2. Reduce Fragmentation

Store files contiguously or perform disk defragmentation.

Result

Fewer disk seeks.

Lower access latency.

Faster file operations.

Q9. Thread Scheduling Debugging (10 Marks)

Problem Identification

5,000 active threads

Memory usage = 92%

Response time = 12 sec

Root Cause

The server creates a new thread for every client request.

Frequent thread creation and destruction consume CPU and memory.

Effect on Performance

High context switching.

Increased memory usage.

Longer scheduling delay.

Slow response time.

Optimizations

1. Use Thread Pool

Reuse existing worker threads.

2. Use Asynchronous/Event-Driven Processing

Reduce the number of active threads.

Result

Lower memory consumption.

Faster response.

Better scalability.

Q10. Disk Scheduling Optimization (10 Marks)

Problem Identification

Current Queue:

98, 183, 37, 122, 14, 124, 65, 67

Head Position:

53

FCFS services requests in arrival order.

Root Cause

FCFS causes unnecessary head movement because requests are processed without considering their location.

This increases seek time and reduces throughput.

Comparison

FCFS SSTF (Shortest Seek Time First)

Processes requests in arrival order Chooses nearest request

High seek time Lower seek time

Low throughput Higher throughput

Slower response Faster response

Optimization

Use SSTF.

Order becomes approximately:

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

This significantly reduces head movement.

Result

Lower seek time.

Higher throughput.

Faster response time.